# Support message

Review requested · Today at 01:02

**You replied**

Case number: 1838100803582037

Hi Meta Security Team,

I am writing to provide a final update regarding Case #1838100803582037. It has been over 10 weeks since my last message and 102 days since my initial report on October 12, 2025.

As I mentioned in my previous correspondence, my priority has always been the resolution and root-cause analysis of the server-side authorization bypass I documented. Since the standard 90-day coordinated disclosure window has now passed and the case remains closed as "Not Applicable" -despite the targeted patching of the accounts I provided - I will be proceeding with a public technical disclosure.

To ensure the security community can fully vet the findings, my disclosure will include:

Complete Technical Logs: Timestamped network captures including X-FB-Debug headers and CDN links exposed during unauthenticated sessions.

Integrity-Verified Evidence: A comprehensive archive of screen recordings with corresponding SHA256 hashes to verify the state of the vulnerability at the time of reporting.

Logic Analysis: A deep dive into the "Header-Trigger" behavior that forced the server to return private polaris_timeline_connection edges to unauthorized requests.

The Patch Timeline: A chronological record showing the immediate hotfix applied to the specific accounts I shared for reproduction.

I remain open to a professional technical discussion regarding the root cause of this bypass. However, if I do not receive a response within 48 hours, I will move forward with publishing the full technical write-up to my research blog and the wider security community.

Best regards,

Jatin

Tuesday 4 November 2025 at 03:52

**You replied**

Case number: 1838100803582037

Hi Meta Security Team,

I'm reaching out one last time to ask you to reopen Case number 1838100803582037. I really hope this gets to someone who can take a fresh look at it, because I feel like the real issue here hasn't been fully explored yet.

Look, from everything you've said in your responses, it seems like the root cause of this vulnerability was never properly diagnosed. You mentioned it might have been fixed as a side effect of some other changes-like a new feature or an infrastructure tweak-but without digging into what actually caused it, how can we be sure it's completely gone for good? I pointed out patterns, like how it only hit certain accounts based on age or other factors, but that wasn't addressed.

To give you more to work with, I'm happy to share the exact network headers from when the exploit worked (showing the polaris_timeline_connection exposure) versus when it didn't on other accounts. I also have the CDN links from those exposures, all timestamped in my logs. Just let me know a secure way to send them over, and I'll do it right away to help pinpoint what's going on.

directly fixed. An unintended patch might not cover all the bases, especially since it was conditional and only showed up under specific setups.

And honestly, since you closed it as "Not Applicable," meaning you don't see it as a real issue, I don't even have to be doing this. But I'm choosing to because user privacy matters to me, and I'd rather work with you to make sure it's sorted. If not, I do have the right to share this responsibly, as per your own assessment that it doesn't exist. But think about it-what if someone else tests this after a disclosure and finds it still affects more accounts we didn't know about? That could be bad for everyone. If you're truly confident it's resolved and no further action is needed, I'll proceed with public disclosure to highlight the concerns. If you close this report again without addressing it, I'll have no other options left but to disclose it publicly, as all my avenues for resolution would be exhausted.

I'd appreciate it if you could reopen the case and do a proper root cause analysis, especially since no direct changes were made based on my report. I'm here to help however I can.

Best regards,

Jatin

Wednesday 29 October 2025 at 17:01

**Our reply**

Case number: 1838100803582037

Hi Jatin,

I understand your frustration, but this issue was unreproducible for our team at the time of the initial investigation as well as after receiving additional details. There are many components of Facebook services that may change over time -- this change could have easily been a side effect of a recently added feature, an unrelated bug fix, or an infrastructure/configuration change.

The decision at the time of the report was that it did not qualify for a reward given no changes were made directly in response at the time.

Although this issue does not qualify for a bounty we appreciate your report. We hope you will continue to report any security issues you notice in the future.

Thanks,

Sparrow

Meta Security

**Your reply**

Write your reply here...

Add Attachment | Send

Monday 27 October 2025 at 18:32

**You replied**

Case number: 1838100803582037

Hi Sarah,

I respectfully disagree with this assessment.

Timeline of Facts:

- Oct 12: I submitted comprehensive report with POC
- Oct 13: You requested I test on 2fa_2fa
- Oct 14: You stated you "cannot reproduce on real aged accounts"
- Oct 14: You specifically requested "another account besides @jatin.py"

The timing is clear: I provided specific vulnerable account names, and those specific accounts were fixed within 48 hours or less. I also gave you a list of vulnerable & non-vulnerable accounts for comparisons.

To claim the issue was "unreproducible" contradicts your own request for additional vulnerable accounts. If it were unreproducible, why did you ask me for more accounts where this was reproduceable?

Additionally, the fix specifically addressed the polaris_timeline_connection exposure I documented. The mobile header trigger behavior (0 follower/following count) persists, proving the fix was targeted at the exact vulnerability I reported, not a broad unrelated change.

I have comprehensive timestamped documentation including:

- Complete ticket correspondence showing your requests for specific accounts

- Network logs with X-FB-Debug headers, CDN links that were exposed.

- Multiple timestamped videos (pre-patch and post-patch) with sha265 fingerprints

- Statistical analysis showing 28% exploitability across "consenting" tested accounts

I request escalation for re-evaluation.

Best regards,

Jatin

Monday 27 October 2025 at 17:13

**Our reply**

Case number: 1838100803582037

Hi Jatin,

Thanks for the update. From our side, the fact that an unreproducible issue was fixed doesn't change the fact that it was not reproducible at the time. Additionally, even if the issue were reproducible, it's possible that a change was made to fix a different issue and this issue was fixed as an unintended side effect.

Although this issue does not qualify for a bounty we appreciate your report. We hope you will continue to report any security issues you notice in the future.

Thanks,

Sarah

Meta Security

Friday 17 October 2025 at 20:06

**You replied**

Case number: 1838100803582037

Hi Meta Security Team,

I hope this message finds you well. I have an important status update.

As of October 16, 2025 , at 6:00 PM IST (12:30 PM UTC), the reported vulnerability is no longer reproducible through my POC script or manual reproduction.

Testing on the same accounts where the server-side authorization bypass was previously working (jatin.py, its_prathambanga) now shows the issue has been resolved. It appears a fix has been successfully deployed.

Could you please confirm the resolution status of this case when convenient? I appreciate any status update you can provide.

Best Regards,

Jatin

Wednesday 15 October 2025 at 20:23

**You replied**

Hi Sarah,

I have a analysis that provides a clear, provable pattern for reproducing this vulnerability.

My investigation reveals a consistent, two-part behavior:

1. The Trigger:

The issue only manifests when a request is sent with the specific mobile headers from my POC script. When this condition is met, "something triggers" on the server, and it always returns a response with incorrect follower_count and following_count (set to 0).

Crucially, this is visible in the browser UI if you are reproducing it manually. The profile will incorrectly show 0 followers and 0 following. This is the fingerprint that confirms you are in the correct state to test for the bug.

2. The Vulnerability:

Once you are in the state described above, the vulnerability itself appears. For a subset of accounts (like jatin.py, its_prathambanga), the server then proceeds to incorrectly populate the polaris_timeline_connection.edges array with private post data. For other accounts (anon_hax0r, 2fa_2fa), this array correctly remains empty.

By comparing what is different between the affected accounts and non affected accounts we can get to the root cause.

Best Regards,

Jatin

Tuesday 14 October 2025 at 20:45

**You replied**

Case number: 1838100803582037

Hi Sarah,

Thank you for your follow-up. I have a critical update that explains the inconsistency we are all observing.

Following your request, I was able to secure explicit written consent from a family member to test their aged, private account. I can confirm the vulnerability is 100% reproducible on this second account.

Attached Evidence:

- Video Proof : A new screen recording showing the successful exploit on this second, consenting account. [https://drive.google.com/file/d/1tu4NZczK_gND6KfTbL6ZHKeV-7eqmGhd/view?usp=sharing]

- Consent Proof : A screenshot of the written consent from the account owner is attached, per your policy.

However, my continued testing has revealed a crucial finding: account age is not the only factor. The vulnerability is more specific than we initially thought.

To give you a complete picture, here is a summary of my test results across all accounts I have access to:

Summary of Findings

Confirmed VULNERABLE Accounts:

jatin.py (Owned by me, Aged)

its_prathambanga (Consenting third-party, Aged)

Confirmed NOT VULNERABLE Accounts:

jatin.bug.py (Owned by me, NEW)

anon_hax0r (Owned by me, Aged)

memez.co.ind (Owned by me, Aged)

2fa_2fa (Meta test account, New)

I trust this provides the definitive evidence you need to confirm the issue and begin a deeper investigation. I am ready to assist in any way I can.

Best Regards,

Jatin

Tuesday 14 October 2025 at 18:07

**Our reply**

Case number: 1838100803582037

Hi Jatin,

We are unable to reproduce this with real accounts that were created several years ago. Are you able to replicate this behaviour with any other account besides @jatin.py? If so, can you please provide the username and a video proof-of-concept?

Thanks,

Sarah

Meta Security

Monday 13 October 2025 at 18:03

**You replied**

Case number: 1838100803582037

Hi Julian,

Apologies for the quick follow-up, but I believe I have made a critical breakthrough in my analysis that may be invaluable to your team.

Since my last reply, I continued to investigate why the vulnerability works on some accounts but not others. My tests now strongly indicate that the vulnerability is conditional and directly tied to account age.

Here is the summary of my findings:

- VULNERABLE: The exploit is consistently reproducible on my established real accounts, which were created several years ago.

- NOT VULNERABLE: The exploit fails on the brand-new account I created today (October 13, 2025) specifically for this investigation.

- NOT VULNERABLE: The exploit fails on your 2fa_2fa test account, which was created recently (September 2025).

I wanted to share this immediately as I believe it is the key to reproducing the issue and will help your engineering team isolate the flaw much more quickly.

Thank you again for your time.

Monday 13 October 2025 at 14:50

**You replied**

Case number: 1838100803582037

Hi Julian,

Thank you for the update and for providing a test case.

I have completed the test you requested. Here are my findings:

- I can confirm that my proof-of-concept did NOT work on the 2fa_2fa account. The polaris_timeline_connection object was present but did not expose any posts.

- To ensure the vulnerability still exists, I immediately re-ran the exact same test on my personal private test account jatin.py and another private test account I own. The vulnerability remains 100% reproducible on these accounts.

To save your team time, here are the two most direct ways to see the vulnerability in action:

- Watch the Attached VideoThe new recording is under 1 minutes and shows the entire process from a poc script:

- The poc.py script I submitted earlier automates everything. It contains all the necessary headers and logic to extract the data. You can test it on a real user and see the results

Monday 13 October 2025 at 12:53

**Our reply**

Case number: 1838100803582037

Hi Jatin,

Thank you for your report.

Can you try to view the private media of our test user?

- 2fa_2fa

Thanks,

Julian

Meta Security

Monday 13 October 2025 at 12:51

**Our reply**

Case number: 1838100803582037

Hi Jatin,

A member of Meta's security team has seen your report and performed an initial evaluation. We will get back to you once we have more information to share.

Thanks,

Meta Security

Sunday 12 October 2025 at 04:23

**Our reply**

Case number: 1838100803582037

Hi,

Thank you for reporting a security issue! Your report number is 1838100803582037. Please give us reasonable time to investigate and mitigate the issue before sharing information with others, and note that we reserve the right to publish your report. (More details: https://www.facebook.com/whitehat/.) Note that if you're writing to us in a language other than English, we'll only be able to respond in English at this time. We're sorry for any inconvenience this may cause.

If you're trying to report another issue, please review the information below to get help.

- If your account or a friend's account is sending out suspicious links: https://www.facebook.com/help/hacked

- To report abuse: https://www.facebook.com/help/reportlinks

- To report bugs that are not security issues: https://www.facebook.com/help/www/326603310765065

- For any other questions or concerns, please visit our Help Center: https://www.facebook.com/help

Thank you,

Meta Security

Sunday 12 October 2025 at 04:23

**What you submitted**

Case number: 1838100803582037

**Title**
Server-Side Authorization Bypass: Instagram Mobile Web Returns Private Timeline Data to Unauthenticated Requests

vulnerability that causes private account timeline data to be included in HTML responses sent to completely unauthenticated users. This is NOT a caching issue - Instagram's backend servers are actively generating and delivering private content to unauthorized requests.

THE VULNERABILITY:

When an unauthenticated user requests a private Instagram profile using mobile headers, Instagram's server responds with HTML that contains embedded JSON data including the private account's complete timeline with direct links to all private posts.

Key technical details:
- This occurs in the INITIAL server response (not cached data)
- The server embeds JSON inside <script type="application/json"> tags in the HTML
- This JSON contains the complete timeline structure: posts, images, metadata
- The vulnerability exists in Instagram's server-side rendering logic for mobile web
- No cookies, authentication tokens, or prior access required

WHY THIS IS NOT A CACHING ISSUE:

I understand your response might be: "CDN/cache limitations are expected behavior."

However, this is fundamentally different:

CACHING ISSUE (not a vulnerability):
- User previously had authorized access to content
- Content was legitimately delivered to their device
- Old cached URLs continue to work after access is revoked
- Expected limitation of CDN architecture

THIS VULNERABILITY (critical authorization bypass):
- User has NEVER had authorized access to the private account
- User has NEVER followed or viewed the account while logged in
- User makes a fresh HTTP request with NO authentication
- Instagram's SERVER actively generates and returns private timeline data
- This happens on EVERY request, not from cache

The server is incorrectly including private data in responses to unauthenticated clients.

TECHNICAL EXPLANATION:

REQUEST (from completely unauthenticated client):
- URL: https://www.instagram.com/<private_username>/
- Method: GET
- Cookies: NONE (fresh incognito browser)
- Authentication: NONE
- Headers: mentioned in poc.py file.

RESPONSE (from Instagram's server):
- HTML document containing embedded JSON
- JSON located in: <script type="application/json" data-content-len="...">
- JSON structure path: require → ScheduledServerJS → __bbox → RelayPrefetchedStreamCache → result → data → xig_user_by_igid_v2 → polaris_timeline_connection
- Contains: Complete array of private posts (edges → nodes)
- Each post includes: Post ID, image URLs (image_versions2), captions, metadata

EXPLOITATION:

- Navigate to polaris_timeline_connection object
- Extract image URLs from image_versions2 arrays
- URLs are directly accessible (CDN links)

This entire process requires ZERO authentication and works on ANY private account.

PROOF OF CONCEPT:

I have created a Python proof-of-concept script that demonstrates this vulnerability:

SCRIPT CONTENTS:
- Constructs HTTP request with mobile headers and NO authentication
- Sends GET request to private Instagram profile
- Receives HTML response from Instagram's server
- Parses HTML to extract embedded JSON script tags
- Locates polaris_timeline_connection structure in JSON
- Extracts all private post image URLs from image_versions2 arrays
- Decodes escaped URLs
- Saves accessible private content URLs to file

The script successfully extracts private post URLs from accounts I have never followed, never viewed, and have no relationship with.

VIDEO DEMONSTRATION:
My attached video shows:
- Starting in fresh incognito browser (no cookies, logged out)
- Opening Chrome DevTools Network tab
- Navigating to a private Instagram account (my own test account)
- Showing the account displays "This Account is Private" to unauthorized users
- Examining the HTML response in Network tab
- Finding the embedded JSON with private timeline data
- Extracting and accessing private post URLs
- All from a completely unauthenticated state

VERIFICATION:

I tested this on my own private Instagram account to ensure ethical testing:
- Created posts on my private account
- Used a fresh browser/device I've never logged into that account on
- Confirmed completely logged out (no cookies)
- Successfully extracted all private post URLs from my account
- Verified URLs are accessible without authentication

SEVERITY: CRITICAL

PRIVACY VIOLATION:
- Complete circumvention of Instagram's private account privacy model
- Any person worldwide can access private posts from ANY private account
- Zero authentication, authorization, or follower relationship required
- Users have no way to detect unauthorized access to their private content
- Affects every private Instagram account (hundreds of millions of users)

USER HARM:
- Private photos and videos users explicitly restricted are publicly accessible
- Personal content (family photos, location data, personal moments) exposed
- No consent or notification when private content is accessed
- Enables stalking, harassment, doxxing, and privacy violations at scale
- Users trust Instagram's "private account" feature - this trust is violated

TECHNICAL IMPACT:
- Server-side authorization bypass in mobile web rendering path

- Affects the core authentication/authorization logic

EXPLOITATION RISK:
- Trivially exploitable with basic HTTP requests
- Can be fully automated with simple scripts
- No sophisticated tools or techniques required
- Scalable to mass data harvesting with residential proxies
- Could be weaponized for:
Mass surveillance of private accounts
Data broker harvesting operations
Stalking and harassment campaigns
Corporate/nation-state espionage
Doxxing operations
CSAM identification/exploitation

**Repro Steps**
1. Prepare fresh environment. Open Chrome in Incognito/Private mode .Navigate to "Network" tab in DevTools

2. Decrease the viewport area by expanding the networks tab area.

3. In the browser address bar, navigate to: instagram.com/<private_username> Replace <private_username> with any private Instagram account username. The page will load showing "This Account is Private" message

4. In the Network tab, locate the request to /<private_username>. Click on this request to view details. Click the "Response" tab to see the HTML returned by Instagram's server. You are now viewing the HTML that Instagram's server generated and sent

5. In the Response view, press Ctrl+F to search. Search for: "polaris_timeline_connection". You will find this inside a <script type="application/json"> tag. This is NOT cached data - this is fresh JSON embedded by Instagram's server

6. Look at the JSON structure containing "polaris_timeline_connection". Navigate to: edges → [array of posts]. Each object in the edges array represents one private post. Look for: node → image_versions2 → candidates
This contains arrays of image URLs in different resolutions

7.
- Copy one of the URLs from the image_versions2 candidates array
- The URLs will be escaped (e.g., "https:\/\/scontent.cdninstagram.com\/...")
- Decode the URL (unescape the \/ characters and \uXXXX sequences)
- Paste the decoded URL in your browser address bar
- The private post image loads successfully WITHOUT any authentication

ATTACHED EVIDENCE:

1. POC.PY
Python script that automates the vulnerability exploitation:
- Sends unauthenticated HTTP request with mobile headers
- Receives HTML response from Instagram server
- Parses embedded JSON from script tags
- Extracts all private post URLs from polaris_timeline_connection
- Decodes escaped URLs
- Saves accessible private content links to file
- Demonstrates the complete attack chain programmatically

2. VIDEO_DEMONSTRATION
Screen recording showing:
- Fresh incognito browser (logged out)

- Complete exploitation from unauthenticated state

Both the script and video prove this is a server-side authorization vulnerability, not a caching issue.

See Options